

SIMATS ENGINEERING

ASSESSMENT TOOL 2-Short Answer Test

COURSE NAME: Data Structures

COURSE CODE: CSA0306

COURSE OUTCOME COVERED

CO: Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem. (Bloom Level -BL4, BL6)

Assessment Tool Weightage: 30 %

Code of Conduct:

I, K. Anil kumar reddy (292525239) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: k. Anil kumar reddy

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	20	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge

		g			
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Anil kumar reddy
(193555239)

CSA0305

DATA STRUCTURE

AT3

- 1A - Acyclic graph
- 2A - Connected graph
- 3A - Adjacency matrix
- 4A - Directed Acyclic Graph (DAG)
- 5A - Directed graph
- 6A - Tree : if the graph remains Connected
- 7A - Directed graph
- 8A - It must be a Directed Acyclic Graph (DAG)
- 9A - Bipartite graph.
- 10A - Tree
- 11A - Directed Acyclic Graph (DAG)
- 12A - The Graph Contains a cycle.
- 13A - Stack.
- 14A - DFS cycle detection or kahn's algorithm
- 15A - Directed by Acyclic Graph (DAG)
- 16A - priority Queue
- 17A - Directed Acyclic Graph (DAG)
- 18A - Prim's Algorithm.
- 19A - Bellman -ford algorithm
- 20A - Breadth -first Search (BFS).

- (21A) Relaxation.
- (22A) Dijkstra's Algorithm.
- (23A) Floyd - Warshall Algorithm
- (24A) $O(V^3)$
- (25A) Distance matrix (2D array)
- (26A) Depth - first Search (DFS)
- (27A) Graph Traversal.
- (28A) Minimum Spanning Tree (MST)
- (29A) It helps obtain a minimum spanning tree by removing an unnecessary high-cost edge.

- 30. Cut property.
- 31. Floyd - Warshall Algorithm.
- 32. Relaxation.
- 33. BFS or DFS
- 34. BFS or DFS
- 35. BFS.
- 36. Node
- 37. BFS-like traversal using a queue.
- 38. DFS-based Topological Sort.
- 39. path.
- 40. Weighted graph.
- 41. NO
- 42. Total edge weight/cost.
- 43. Connected, weighted, undirected graph.
- 44. Dijkstra's Algorithm.
- 45. A* (A-Star) Algorithm.

- 46A. Dijkstra's Algorithm.
- 47A. shortest distance/path.
- 48A. dynamic programming.
- 49A. All-pairs shortest paths.
- 50A. Bellman-Ford Algorithm.
- 51A. $n(n-1)/2$.
- 52A. NO
- 53A. Kruskal's Algorithm.
- 54A. $V-1$ edges.
- 55A. A location intersection/city.
- 56A. Floyd-Warshall Algorithm.
- 57A. The MST remains the same.
- 58A. YES.
- 59A. Adjacency matrix.
- 60A. All edges weight are non-negative.
- 61A. An estimated cost/distance from the current node to the goal.
- 62A. All edge weights are distinct.
- 63A. if $d[u] > d[v] + W(u,v)$, update $d[v] = d[u] + W(u,v)$.
- 64A. A negative value ($d[i][i] < 0$).
- 65A. BFS or DFS
- 66A. A source node.
- 67A. Bellman-Ford.
- 68A. $O(V^2)$
- 69A. An edge to a currently visiting/ancestor vertex.
- 70A. The MST cost is a lower bound on the cost of a tour.

- 71A. BFS minimizes number of edges, not total edge weight.
- 72A. Prim's and Kruskal's algorithms.
- 73A. Negative all edge weights and find the shortest path.
- 74A. Resource Allocation Graph.
- 75A. Minimum number of connections / degree of separation.
- 76A. A complete topological ordering is impossible.
- 77A. Disjoint Set Union (DSU) / Union-Find.
- 78A. Its $O(N^3)$ time complexity is expensive for large or frequently changing graphs.
- 79A. It makes future Find operations faster by shortening the tree paths.
- 80A. No, if edge weights are non-negative; the ordering of weights remains unchanged.
- 81A. Path Vector Algorithm.
- 82A. Tasks with no outgoing dependencies, usually final tasks.
- 83A. A walk may repeat vertices/edges; a path does not repeat vertices.
- 84A. $n-2$.
- 85A. Fuel consumption / cost for travelling along the road.
- 86A. Bellman-Ford Algorithm.
- 87A. A previously finalized shortest distance can later be reduced by a negative edge.

88A. It is the allowed intermediate vertex used to improve the path from i to j .

89A. zero; a bipartite graph contains no odd cycles.

90A. DFS. Commonly preorder traversal.

91A. 2, if the path is viewed as a DAG with edges oriented consistently along the path.

92A. Number of incoming edges / dependencies of the vertex.

93A. To select the smallest-weight edge first.

94A. Color vertices with two colours during BFS; if adjacent vertices get the same color, it is not bipartite.

95A. A maximal set of vertices where every vertex is reachable from every other vertex.

96A. Floyd-Warshall can find all pairs shortest paths, then take the maximum finite distance.

97A. Maximum amount of water that can flow through the pipe.

98A. Always choose the minimum-weight edge connecting the growing MST to an unvisited vertex.

99A. Adjacency list.

100A. The direct distance b/w two points is no greater than the distance through an intermediate point:
$$d(A, C) \leq d(A, B) + d(B, C).$$